



Making Data Structures Persistent

James R. Driscoll

Comp. Sci. Dept., Carnegie-Mellon Univ.
Pittsburgh, PA 15218

Neil Sarnak

Courant Inst. of Mathematical Sciences
New York University
New York, New York 10012

Daniel D. Sleator

Comp. Sci. Dept., Carnegie-Mellon Univ.
Pittsburgh, PA 15218

Robert E. Tarjan

Comp. Science Dept., Princeton Univ.
Princeton, New Jersey 08544
and
AT&T Bell Laboratories
Murray Hill, NJ 07974

ABSTRACT

This paper is a study of *persistence* in data structures. Ordinary data structures are *ephemeral* in the sense that a change to the structure destroys the old version, leaving only the new version available for use. In contrast, a persistent structure allows access to any version, old or new, at any time. We develop simple, systematic, and efficient techniques for making linked data structures persistent. We use our techniques to devise persistent forms of binary search trees with logarithmic access, insertion, and deletion time and $O(1)$ space bounds for insertion and deletion.

1. Introduction

Ordinary data structures are *ephemeral* in the sense that making a change to the structure destroys the old version, leaving only the new one. However, in a variety of problems in such areas as computational geometry [6,9,11,24,25,28,30], text and file editing [26], and implementation of very high level programming languages [18], multiple versions of a data structure must be maintained. We shall call a data structure *persistent* if it supports access to multiple versions. The structure is *partially persistent* if all versions can be accessed but

only the newest version can be modified, and *fully persistent* if every version can be both accessed and modified.

A number of researchers have devised partially or fully persistent forms of various data structures, including stacks [21], queues [13], search trees [18,20,22,24,26,30], and related structures [6,9,11]. Most of these results use ad hoc constructions; with the exception of one paper by Overmars [25], discussed in Section 2, there has been no systematic study of persistence. Providing such a study is our purpose in this paper, which is an outgrowth of the second author's Ph.D. thesis [27].

We shall discuss generic techniques for making linked data structures persistent at small cost in time and space efficiency. Since we want to take a general approach, we need to specify exactly what a linked structure is and what kinds of operations are allowed on the

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

structure. Formally, we define a *linked data structure* to be a finite collection of *nodes*, each containing a fixed number of named *fields*. Each field is either an *information field*, able to hold a single piece of information of a specified type, such as a bit, an integer, or a real number, or a *pointer field*, able to hold a pointer to a node or the special value *null* indicating no node. We shall assume that all nodes in a structure are of exactly the same type, i.e. have exactly the same fields; our results easily generalize to allow a fixed number of different node types. Access to a linked structure is provided by a fixed number of named *access pointers* indicating nodes of the structure, called *entry nodes*. We can think of a linked structure as a labeled directed graph whose vertices have constant out-degree. If a node x contains a pointer to a node y , we call y a *successor* of x and x a *predecessor* of y .

As a running example throughout this paper, we shall use the binary search tree. A *binary search tree* is a binary tree* containing in its nodes distinct items selected from a totally ordered set, one item per node, with the items arranged in symmetric order: if x is any node, the item in x is greater than all items in the left subtree of x and less than all items in the right subtree of x . A binary search tree can be represented by a linked structure in which each node contains three fields: an *item* (information) field and *left* and *right* (pointer) fields containing pointers to the left and right children of the node. The tree root is the only entry node.

On a general linked data structure we allow two kinds of operations: *access operations* and *update operations*. An access operation consists of a sequence of *access steps*, which compute a set of *accessed nodes* by beginning with the empty set and adding nodes. An access step consists of adding one node to the accessed set. This node must either be an entry node or be indicated by a pointer in a previously accessed node. The time of an access operation is the number of access steps performed. In an actual data structure the successively accessed nodes would be determined by examining the fields of previously accessed nodes, and the access operation would produce as output some of the information contained in the accessed nodes, but we shall not be concerned with the details of this process. An example of an access operation is a search for an item in a binary search tree. The accessed set forms a path in the tree that starts at the root and is extended one node at a time until the desired item is found or a null pointer, indicating a missing node, is reached.

An *update operation* consists of an intermixed sequence of access steps and *update steps*. The access steps compute a set of accessed nodes as described above. The update steps change the structure. An update step consists either of creating a new node, which is added to the accessed set changing a single field in an accessed node, or changing an access pointer. If a pointer field or

an access pointer is changed, the new pointer must indicate a node in the accessed set or be null. A newly created node must have its information fields explicitly initialized; its pointer fields are initially null. As in the case of an access operation, we shall not be concerned with the details of how the steps to be performed are determined. The *total time* of an update operation is the total number of access and update steps; the *update time* is the number of update steps. If a node is not indicated by any pointer in the structure, it disappears from the structure; that is, we do not require explicit deallocation of nodes.

An example of an update operation is an insertion of a new item in a binary search tree. The insertion consists of a search for the item to be inserted, followed by a replacement of the missing node reached by the search with a new node containing the new item. A more complicated update operation is a deletion of an item. The deletion process consists of three parts. First, a search for the item is performed. Second, if the node containing the item has a left child, the item is swapped with the item preceding it in symmetric order, found by starting at the left child of the node containing the item and following successive right pointers until reaching a node with no right child. Now the item to be deleted is guaranteed to be in a node with only one child. The third part of the deletion consists of removing the node containing the item from the tree and replacing it by its only child, if any. The total time of either an insertion or a deletion is the depth of some tree node plus a constant; the update time is only a constant.

Returning to the case of a general linked data structure, let us consider a sequence of intermixed access and update operations on an initially empty structure (one in which all access pointers are null). We shall denote by m the total number of update operations. We index the update operations and the versions of the structure they produce by integers: update i is the i^{th} update in the sequence; version 0 is the initial (empty) version, and version i is the version produced by update i . We are generally interested in performing the operations on-line; that is, each successive operation must be completed before the next one is known.

We can characterize ephemeral and persistent data structures based on the allowed kinds of operation sequences. An ephemeral structure supports only sequences in which each successive operation applies to the most recent version. A partially persistent structure supports only sequences in which each update applies to the most recent version (update i applies to version $i-1$), but accesses can apply to any previously existing version (whose index must be specified). A fully persistent structure supports any sequence in which each operation applies to any previously existing version. For any of these kinds of structure, we shall call the operation being performed the *current operation* and the version to which it applies the *current version*. The current version is *not* necessarily the same as the newest version (except in the case of an ephemeral structure.) We denote by n the

* Our tree terminology is that of [31].

number of nodes in the current version.

The problem we wish to address is as follows. Suppose we are given an ephemeral structure; that is, we are given implementations of the various kinds of operations allowed on the structure. We want to make the structure persistent; that is, to allow the operations to occur in the more general kinds of sequences described above. In an ephemeral structure only one version exists at any time. Making the structure persistent requires building a data structure representing all versions simultaneously, thereby permitting access and possibly update operation to occur in any version at any time. This data structure will consist of a linked structure (or possibly something more general) with each version of the ephemeral structure embedded in it, so that each access or update step in a version of the ephemeral structure can be simulated (ideally in $O(1)$ time) in the corresponding part of the persistent structure.

The main results of this paper are in Sections 2-5. In Section 2 we discuss how to build partially persistent structures, which support access but not update operations in old versions. We show that if an ephemeral structure has nodes of bounded in-degree, then the structure can be made partially persistent at an amortized* space cost of $O(1)$ per update step and a constant factor in the amortized time per operation. The construction is quite simple. Using more powerful techniques we show in Section 3 how to build a fully persistent version of an ephemeral structure with nodes of bounded in-degree. As in Section 2, the amortized space cost is $O(1)$ per update step and the amortized time cost is a constant factor.

In Sections 4 and 5 we focus on the question of making balanced search trees fully persistent. In Section 4 we show how the result of Section 2 provides a simple way to build a partially persistent balanced search tree with a worst-case time per operation of $O(\log n)$ and an amortized space cost of $O(1)$ per insertion or deletion. We also combine the result of Section 3 with a technique of Tsakalidis [35,36] for delayed updating to obtain a fully persistent form of balanced search tree with the same time and space bounds as in the partially persistent case. In Section 5 we use another technique to make the $O(1)$ space bound for insertion and deletion worst-case instead of amortized.

Section 6 is concerned with applications, extensions, and open problems. The partially persistent balanced search trees developed in Section 4 have a variety of uses in geometric retrieval problems, a subject more fully discussed in the companion paper [28]. The fully persistent balanced search trees developed in Sections 4 and 5 can be used in the implementation of very high level programming languages, as can the fully persistent deques obtainable from the results of Section 3.

In this shortened version of the paper we omit many of the details of our ideas.

* By *amortized cost* we mean roughly the cost of an operation averaged over a worst-case sequence of operations. See [33].

2. Partial Persistence

We begin our study by considering how to make an ephemeral data structure partially persistent. Recall that partial persistence means that each update applies to the newest version, i.e. version i is formed by updating version $i-1$. We shall begin by reviewing the results of Overmars [25], who studied three simple but general ways to obtain partial persistence. One method is to explicitly store every version, copying the entire ephemeral structure after each update. This costs $\Omega(n)$ time and space per update. An alternative method is to store no versions but instead to store the entire update sequence, rebuilding the current version from scratch each time an access is performed. If storing an update takes $O(1)$ space, this method uses only $O(m)$ space, but accessing version i takes $\Omega(i)$ time even if a single update takes $O(1)$ time. A hybrid method is to store the entire update sequence and in addition every k^{th} version, for some suitably chosen value of k . Accessing version i requires rebuilding it from version $k \lfloor i/k \rfloor$ by performing the appropriate sequence of updates. This method has a time-space tradeoff that depends on k and on the running times of the ephemeral access and update operations. Unfortunately any choice of k causes a blow-up in either the storage space or the access time by a factor of $\sqrt{m}$, if one makes reasonable assumptions about the efficiency of the ephemeral operations.

The third approach of Overmars is to use the dynamization techniques of Bentley and Saxe [2], which apply to so-called "decomposable" searching problems. Given an ephemeral data structure representing a set of items, on which the only update operation is insertion, the conversion to a persistent structure causes both the access time and the space usage to blow up by a logarithmic factor, again if one makes reasonable assumptions about the efficiency of the ephemeral operations. If deletions are allowed the blow-up is much greater.

We seek more efficient techniques. Ideally we would like the storage space used by the persistent structure to be $O(1)$ per update step and the time per operation to increase by only a constant factor over the time in the ephemeral structure. One reason the results of Overmars are so poor is that he assumes very little about the underlying ephemeral structure. By restricting our attention to linked structures and focusing on the details of the update steps, we are able to obtain much better results.

Our first idea is to record all updates to node fields in the nodes themselves, without erasing old values of the fields. This requires that we allow nodes to become arbitrarily "fat", i.e. to hold an arbitrary number of values of each field. Using this idea we simulate update steps on the ephemeral structure as follows. When an ephemeral update step creates a new node, we create a corresponding new fat node. When an ephemeral update step changes a field value in a node, we add the new value to the corresponding fat node, along with the name of the field whose value has changed and a *version stamp* that is the number of the update operation being performed.

That is, changes made during update i have a version stamp of i . For each field in a node, we store only one value per version; when storing a field value, if there is already a value of the same field with the same version stamp we overwrite the old value.

The resulting persistent structure has all versions of the ephemeral structure embedded in it. We navigate through the persistent structure as follows. When an ephemeral access step applied to version i follows the pointer in field f of a node, we follow the pointer in the corresponding fat node whose field name is f ; if there are several such pointers, we select the one with maximum version stamp no greater than i . The lack of any such pointer in the fat node indicates that the pointer to be followed is null.

We also need an auxiliary data structure to store the access pointers of the various versions. This structure consists of an array of pointers for each access pointer name. After update operation i , we store the current values of the access pointers in the i^{th} positions of these access arrays. With this structure, following an access pointer into any version takes $O(1)$ time.

The fat node method applies to any linked data structure and uses only $O(1)$ space per ephemeral update step. The method has two related drawbacks. First, the fat nodes must be represented by linked collections of fixed-size nodes. This poses no fundamental difficulty but complicates the implementation. Second, choosing which pointer in a fat node to follow when simulating an access step takes more than constant time. If the pointers within a fat node are ordered by version stamp and stored in a binary search tree, simulating an ephemeral access step takes $O(\log m)$ time. This means that there is a logarithmic factor blow-up in the times of access and update operations over their times in the ephemeral structure.

Remark. Although fat nodes must in general be able to hold arbitrarily large numbers of pointers, this is not true in the binary tree application discussed above if insertion is the only update operation. In this case each "fat" node only needs to hold two pointers, one left pointer and one right pointer, each with a version stamp. $\square$

We eliminate the drawbacks of fat nodes with our second idea, *node copying*. We do not allow nodes in the persistent structure to become arbitrarily fat. Instead, we allow each node to hold only a fixed number of pointers. When we run out of space in a node for new pointers, we create a new copy of the node, containing only the current field values. We must also store, in every current predecessor of the node being copied, a pointer to the new copy. If there is no space in a predecessor for such a pointer, the predecessor, too, must be copied. Thus node copying can ripple backwards through the structure, and a single update step can cause an arbitrarily large number of nodes to be copied. Nevertheless, if we assume that the ephemeral structure has nodes of constant bounded in-degree, and we allow sufficient extra space in each node of the persistent structure, then we

can derive an $O(1)$ amortized bound on the number of nodes copied per update step.

In order to develop the details of this idea, it is useful to have a little terminology. We shall call a node of the ephemeral structure being simulated an *ephemeral node* and a node of the persistent structure a *persistent node*. If x is an ephemeral node occurring in version i of the ephemeral structure, version i of x is x together with all its field values in version i of the structure. That is, we think of an ephemeral node as going through several versions as its fields are changed. Let d be the number of pointers in an ephemeral node and let p be the maximum number of predecessors of an ephemeral node in any single version. We assume that p is a constant. Each persistent node contains $d + p + e + 1$ pointer fields, where e is a sufficiently large constant, to be determined later. Of these fields, d are the same as the pointer fields of an ephemeral node and contain *original pointers*, p are for *predecessor pointers*, e are for *extra pointers*, and one is for a *copy pointer*. Each persistent node has the same information fields as an ephemeral node, and in addition a version stamp for the node itself and a field name and a version stamp for each extra pointer.

The correspondence between the ephemeral structure and the persistent structure is as follows. Each ephemeral node x corresponds to a set $F(x)$ of persistent nodes, called a *family*. The members of $F(x)$ form a singly-linked list, linked by the copy pointers, with the newest member last on the list. We call this last member *live* and the other members of $F(x)$ *dead*. Each version of x corresponds to one member of $F(x)$, although several versions of x may correspond to the same member of $F(x)$. The newest version of x corresponds to the live member of $F(x)$. The live nodes and their newest field values represent the newest version of the ephemeral structure. We call a pointer in the persistent structure representing a pointer in the newest version of the ephemeral structure a *live pointer*. To facilitate node copying, each live pointer in the persistent structure has a corresponding *inverse pointer*, i.e. if live node x contains a live pointer to live node y , then y contains a pointer to x stored in one of its e predecessor fields.

Navigation through the persistent structure is exactly the same as in the fat node method. As in that method we use access arrays, one per access pointer name, to store the various versions of the access pointers. Thus we can access any entry node in any version in $O(1)$ time. To simulate an ephemeral access step that applies to version i and follows the pointer in field f of an ephemeral node x , we follow the pointer with field name f in the corresponding persistent node, selecting among several such pointers the one with maximum version stamp no greater than i . (We regard an original pointer in a node as having a version stamp equal to that of the node itself.) Simulating an ephemeral access step in the persistent structure takes $O(1)$ time.

When simulating an ephemeral update operation,

we maintain a set S of nodes that have been copied. Consider update operation i . We begin the simulation of this operation by initializing S to be empty. We simulate ephemeral access steps as described above. When an ephemeral update step creates a new node, we create a corresponding new persistent node with a version stamp of i and all original pointers null. When an ephemeral update step changes an information field in an ephemeral node x , we inspect the corresponding persistent node $\bar{x}$. If $\bar{x}$ has version stamp i , we merely change the appropriate field in $\bar{x}$. If $\bar{x}$ has version stamp i less than i , but has a copy $c(\bar{x})$ (which must have version stamp i), we change the appropriate field in $c(\bar{x})$. If $\bar{x}$ has version stamp less than i but has no copy, we create a copy $c(\bar{x})$ of $\bar{x}$, make the copy pointer of $\bar{x}$ point to it, and fill it with the most recent values of the information fields of x , which excluding the new value of the changed field can be obtained from $\bar{x}$. We also add to $c(\bar{x})$ pointers corresponding to the most recent values of the pointer fields of x . This requires updating inverse pointers and is done as follows. Suppose that $\bar{x}$ contains a pointer to a node $\bar{y}$ as the most recent version of field f . We store in original pointer field f of node $c(\bar{x})$ a pointer to $\bar{y}$, or to the copy $c(\bar{y})$ of $\bar{y}$ if $\bar{y}$ has been copied. We erase the pointer to $\bar{x}$ in one of the predecessor fields of $\bar{y}$, and we store a pointer to $c(\bar{x})$ in the predecessor field of $\bar{y}$ or $c(\bar{y})$ as appropriate. Once $c(\bar{x})$ has all its original pointer fields filled, we add $\bar{x}$ to the set S of copied nodes.

Simulating an ephemeral update step that changes a pointer field is much like simulating a step that changes an information field. If x is the ephemeral node in which the change takes place, we inspect the corresponding persistent node $\bar{x}$. If $\bar{x}$ has version stamp i , we merely change the appropriate original pointer field in $\bar{x}$. If $\bar{x}$ has version stamp less than i but has a copy $c(\bar{x})$, we change the appropriate original pointer field in $c(\bar{x})$. If $\bar{x}$ has a version stamp less than i but has no copy, we check whether $\bar{x}$ has space for an extra pointer. If so, we store the appropriate new pointer in $\bar{x}$, along with the appropriate field name and a version stamp of i . If not, we create a new copy $c(\bar{x})$ of $\bar{x}$, fill it as described above, and add $\bar{x}$ to S . During the simulation, whenever we install a pointer in a persistent node $\bar{x}$, we make sure it points to a live node. More precisely, if the pointer indicates $\bar{y}$ but $\bar{y}$ has a copy $c(\bar{y})$, we place in $\bar{x}$ a pointer to $c(\bar{y})$ instead of $\bar{y}$. Also, whenever installing a new pointer, we update inverse pointers appropriately.

After simulating an entire update operation, we must perform some postprocessing on the set S to make live pointers point to live nodes. This postprocessing consists of repeating the following step until S is empty:

Update Pointers. Remove any node $\bar{y}$ from S . For each node $\bar{x}$ indicated by a predecessor pointer in $\bar{y}$, find in $\bar{x}$ the live pointer to $\bar{y}$. If this pointer has version stamp equal to i , replace it by a pointer to $c(\bar{y})$. If the pointer has version stamp less than i , add a version i pointer from $\bar{x}$ to $c(\bar{y})$, copying $\bar{x}$ as described above if there is no

space for this pointer in $\bar{x}$. If $\bar{x}$ is copied, add $\bar{x}$ to S . When installing pointers, update inverse pointers appropriately.

This completes our description of the persistent structure and the operations on it. As an example of the use of the node copying method, let us apply it to binary search trees. In this application a major simplification is possible: we can dispense with inverse pointers in the persistent structure, because in the ephemeral structure there is only one access path to any node, and accessing a node requires accessing its predecessor. In general we can avoid the use of inverse pointers if in the ephemeral structure a node is only accessed after *all* its predecessors are accessed. In the binary tree case we can also dispense with the copy pointers, since node copying proceeds in a very controlled way, back through ancestors of the node where a change occurs.

It remains for us to analyze the space needed for the persistent structure and the time needed for update steps. We shall derive an amortized $O(1)$ bound on the space and time per update step using the "potential" technique [33]. To obtain this bound, it suffices to choose e to be any integer constant such that $e \geq p$.

Recall that a persistent node is live if it has not been copied and dead otherwise. All updates to fields are in live nodes, except for erasures of inverse pointers, which can occur in dead nodes. We define the *potential* of the persistent structure to be $e/(e-p+1)$ times the number of live nodes it contains minus $1/(e-p+1)$ times the number of unused extra pointer fields in live nodes. Observe that the potential of the initial (empty) structure is zero and the potential is always non-negative, since any node has at most e unused extra pointer fields. We define the *amortized space cost* of an update operation to be the number of nodes it creates plus the net increase in potential it causes. With this definition, the total number of nodes created by a sequence of update operations equals the total amortized space cost minus the net increase in potential over the sequence. Since the initial potential is zero and the final potential is non-negative, the net potential increase over any sequence is non-negative, which implies that the total amortized space cost is an upper bound on the total number of nodes created. A straightforward computation, omitted in this short version of the paper, shows that the amortized space cost of an update operation is linear in the number of update steps. The same analysis shows that the amortized time per update step is also $O(1)$.

3. Full Persistence

Let us now turn to the harder task of making an ephemeral structure fully persistent. The first problem we encounter is that whereas the various versions of a partially persistent structure have a natural linear ordering, the versions of a fully persistent structure are only partially ordered. The partial ordering is defined by a rooted *version tree*, whose nodes are the versions (0 through m), with version i the parent of version j

(denoted $i = p(j)$) if version j is obtained by updating version i . Version 0 is the root of the version tree. The sequence of updates giving rise to version i corresponds to the path in the version tree from the root to i .

Our plan is to obtain full persistence first with the fat node approach of Section 2 and then with a variant of node copying. Unfortunately, the lack of a linear ordering on versions makes navigation through the resulting persistent structure problematic. To eliminate this difficulty, we impose a total ordering on the versions consistent with the partial ordering defined by the version tree. We represent this total ordering by a list of the versions in the appropriate order. We call this the *version list*. When a new version, say i , is created, we insert i in the version list immediately after its parent $p(i)$ in the version tree. The resulting list defines a preorder on the version tree, as can easily be proved by induction. This implies that the version list has the following crucial property: for any version i , the descendants of i in the version tree occur consecutively in their version list, starting with i .

In addition to performing insertions in the version list, we need to be able to answer order queries of the following kind: given two versions i and j , determine whether i precedes or follows j in the version list. Tsakalidis [34], building on the ideas of Dietz [10], and Sleator [29] have proposed list representations supporting such queries in $O(1)$ worst-case time, with an $O(1)$ amortized time bound for insertion. We can use either of these data structures to represent the version list; that of Sleator is substantially simpler.

Having dealt in a preliminary way with the navigation issue, let us investigate how to use fat nodes to make a linked structure fully persistent. We shall use a slightly different node organization than the one used in Section 2, since it simplifies the discussion somewhat. Each fat node is composed of an arbitrarily large number of *version records*, each of which has a version stamp and a set of fields that are the same as the fields of an ephemeral node. If $i_1, i_2, \dots, i_k$ are the version stamps of the version records in some fat node $\bar{x}$, ordered by their positions in the version list, then the version record with stamp i_j contains the field values valid for versions between i_j and i_{j+1} in the version list, including i_j but excluding i_{j+1} . Navigation through the persistent structure is the same in the fully persistent case as in the partially persistent case, except that we compare versions with respect to their positions in the version list rather than with respect to their numeric values. More precisely, to find the value corresponding to that of field f in version i of ephemeral node x , we find in the fat node $\bar{x}$ corresponding to x the version record whose version stamp is i or follows i in the version list, choosing among several candidates the one with version stamp closest to i in the version list. We use the value of field f in this record.

In order to maintain the invariant that node records are valid over intervals of the version list, we must modify slightly the way updates are performed. We

begin update operation i by adding i to the version list as described above. We simulate the update steps of the operation as follows. Consider an update to field f of ephemeral node x . Let $\bar{x}$ be the fat node corresponding to x . If $\bar{x}$ contains a version record with version stamp i , we merely make the appropriate change in this record. Otherwise, let j be the stamp of the version record in $\bar{x}$ that follows i but is closest to it in the version list ordering. We add to node $\bar{x}$ two new version records: one whose version stamp is the successor of i in the version list, containing exactly the same field values as the record of stamp j , and one with stamp i , containing the same field values as the record of stamp j except for the new value of the changed field. If there is already a version record whose stamp is the successor of i , we do not add a new one; we merely leave the old one in place, without updating it. This form of updating maintains the desired interval property.

There are a few more details of the fat node method to be filled in. Whenever an update step in update operation i creates a new node, we create a new fat node with one version record, having stamp i . As in the case of partial persistence we maintain access arrays containing the various values of the access pointers.

The efficiency of the fat node method is the same for full persistence as it is for partial persistence: the space cost per update step is $O(1)$ and the time cost per access step is $O(\log m)$, if a fat node is represented as a search tree of version records, ordered by version stamp. As in the partially persistent case, the method is applicable even if there is no fixed bound on the in-degree of an ephemeral node. A more accurate bound on the time cost of an access step is $O(\log h)$, where h is the maximum number of changes made to an ephemeral node.

Our next topic is the use of a variant of node copying in obtaining full persistence. We shall call the variant *node splitting*, since it resembles the node splitting used to perform insertions in B -trees [1]. As in Section 2, let d be the number of pointers in an ephemeral node and let p be a constant upper bound on the in-degree of an ephemeral node. We shall allow each persistent node to hold up to v version records, where v is a sufficiently large constant, to be chosen later. Each version record in a node x holds, in addition to the fields of an ephemeral node, a set of p *inverse pointers*, indicating those nodes containing pointers to x valid in the versions for which the versions record is valid.

Ignoring the issue of node splitting for the moment, let us see how inverse pointers are handled. Suppose we wish to simulate an ephemeral update step that changes pointer field f in version i of node x to point to node y . Let $\bar{x}$ and $\bar{y}$ be the persistent nodes corresponding to x and y , respectively. We add to both $\bar{x}$ and $\bar{y}$ new version records with version stamp i , containing the appropriately updated field values and inverse pointers. If either $\bar{x}$ or $\bar{y}$ already contains a version record with version stamp i , we merely update the appropriate fields in the already existing record. We also add to each of $\bar{x}$ and $\bar{y}$ a version

record whose version stamp is the successor of i in the version list, containing the same field values as the record in the same node whose version stamp precedes i but is closest to it in the version list. (If such a record already exists in $\bar{x}$ or $\bar{y}$, we do not add a new one.) In this way we are able to maintain the validity of version records, including the inverse pointers they contain, over intervals of versions in the version list. The time to simulate an ephemeral update step is $O(1)$, not counting node splitting.

We must now deal with what happens when a node does not have space for the new one or two version records we wish to store in it. In this case we split the node in two, putting $\lfloor f/2 + 1 \rfloor$ of the version records in one copy and the remainder in the other. To be more precise, we leave the first $\lfloor f/2 + 1 \rfloor$ version records in the original node and we create a new node containing the last $\lfloor f/2 + 1 \rfloor$ or fewer version records. (The ordering is by position of the version stamps in the version list.)

Some of the nodes that contain pointers to the node that split must be changed so that they contain pointers to the new copy. Let $\bar{x}$ be the node that splits and let $\bar{x}'$ be the new copy. Let i be the first version stamp (in version list order) of a version record moved to $\bar{x}'$. If a node $\bar{y}$ contains a version record r with a pointer to $\bar{x}$, this pointer must be changed to indicate $\bar{x}'$ if the version stamp of r equals i or follows i in the version list. If the version stamp precedes i but the next version stamp of a version record in $\bar{y}$ follows i , then a new version record with stamp i containing a pointer to $\bar{x}'$ must be added to $\bar{y}$. If $\bar{y}$ does not contain enough space for a new version record, $\bar{y}$ must be split. Thus splitting can cascade through the structure.

An analysis like that sketched in Section 2 shows that the amortized space cost of an update step is $O(1)$ if v is chosen such that $v \leq 2(d + p + 1)$. Implementing the cascading node splitting process poses some difficulties in implementation and analysis not arising in the node copying method discussed in Section 2. The amortized time per node split is $O(1)$ with an appropriate implementation, which we omit in this short version of the paper.

In summary, the node splitting method will make a linked structure of constant bounded in-degree fully persistent at an amortized space and time cost of $O(1)$ per update step and a worst case time of $O(1)$ per access step. We close this section with two observations. First, the method proposed here can be modified so that changes are associated with individual fields as in Section 2, rather than with complete sets of fields. (Each new field value then has its own version stamp.) Whether this saves space depends on the internal structure of nodes. Second, our cascading node splitting process is formally equivalent to the *fractional cascading* method of Chazelle and Guibas [7,8], which was invented for an entirely different purpose. Indeed, our ideas can be used to develop a slightly simplified form of fractional cascading.

4. Persistent Search Trees

In this section we shall focus on the question of making a specific data structure, namely a balanced search tree, persistent. We shall discuss a particular kind of search tree, the red-black tree, although our ideas apply as well to certain other kinds of search trees. A *red-black tree* [12,31,32] is a binary search tree, each node of which is colored either *red* or *black*, subject to the following three constraints:

- (i) (Missing node convention) Every missing (external) node is regarded as being black.
- (ii) (Red constraint) Every red node has a black parent or has no parent, i.e. is the root.
- (iii) (Black constraint) From any node, all paths to a missing node contain the same number of black nodes.

The color constraints imply that the depth of an n -node red-black tree is at most $2 \log n$ and hence that the time to access any item is $O(\log n)$. To maintain the color constraints, a red-black tree is rebalanced after each insertion or deletion. Rebalancing requires recoloring certain nodes and performing local transformations called *rotations*, each of which preserves the symmetric order of the items in the nodes but changes the depths of some of them, while taking $O(1)$ time.

A bottom-up rebalancing strategy [31,32] leads to especially efficient insertion and deletion algorithms. To perform an insertion, we follow the access path for the item to be inserted until reaching a missing node. At the location of the missing node we insert a new node containing the new item. We color the new node red. This may violate the red constraint, since the parent of the new node may be red. In this case we bubble the violation up the tree by repeatedly applying the recoloring transformation of Figure 1(a) until it no longer applies. This either eliminates the violation or produces a situation in which one of the transformations in Figures 1(b), 1(c), and 1(d) applies, all of which leave no violation.

A deletion is similar. We first search for the item to be deleted. If it is in a node with a left child, we swap the item with the item preceding it in symmetric order, which we find by starting at the left child and following successive right pointers until reaching a node with no right child. Now the item to be deleted is in a node with at most one child. We replace this node by its only child (if any). This does not affect the red constraint but will violate the black constraint if the deleted node is black. If there is a violation the replacing node (which may be missing) is *short*: paths from it to missing nodes contain one fewer black node than paths from its sibling. We bubble the shortness up the tree by repeating the recoloring transformation of Figure 2(a) until it no longer applies. Then we perform the transformation in Figure 2(b) if it applies, followed if necessary by one application of 2(c), (d), or (e).

An insertion requires $O(\log n)$ recolorings plus at most two rotations; a deletion, $O(\log n)$ recolorings plus

at most three rotations. Furthermore the amortized number of recolorings per update operation is $O(1)$, i.e. m update operations require $O(m)$ recolorings [14,15,19].

We can make a red-black tree partially persistent by using the node copying method of Section 2 with e (the number of extra pointers per node) equal to one. Because in the ephemeral structure there is only one access path to any node, we do not need inverse pointers. We obtain an additional simplification because colors are not used in access operations. Thus we do not need to save old node colors but can instead overwrite them. It is also easy to dispense with the copy pointers and the version stamps of the nodes. Thus each persistent node contains an item, three pointers, a color bit, and a version stamp for the third (extra) pointer.

A partially persistent red-black tree representing a history of m insertions and deletions occupies $O(m)$ space and allows $O(\log n)$ -time access to any version, where n is the number of items in the accessed version. The worst-case time per insertion or deletion is also $O(\log n)$. These trees and their applications, of which there are a number in computational geometry, are discussed more fully in the companion paper [28].

The node copying method applies more generally to give an $O(m)$ -space partially persistent form of any balanced search tree with an $O(1)$ amortized update time for insertion and deletion. Such trees include weight-balanced trees [3,23] and weak B -trees [14,15,19]. Increasing the number of extra pointers per node allows the addition of auxiliary pointers such as parent pointers and level links [4,15]. Thus we can obtain a partially persistent form of finger search tree [4,15,16,17,35,36] occupying $O(m)$ space, with only a constant factor blow-up in the amortized access and update times over their ephemeral counterparts.

Let us turn to the problem of making a red-black tree fully persistent. We would like to obtain an $O(m)$ -space fully persistent red-black tree by using the node splitting method of Section 3. There are two difficulties, however. First, we must save old color bits; we are not free to overwrite them. Second, if we include recoloring time the $O(1)$ update time bound for ephemeral insertion and deletion is only amortized, whereas we need a worst-case $O(1)$ bound. Thus our goal becomes the construction of an ephemeral variant of red-black trees in which the worst case update time per insertion or deletion, including recoloring, is $O(1)$.

To obtain such a variant, we use an idea of Tsakalidis [35,36], *lazy recoloring*. We observe that the recoloring that follows an insertion takes place along a single path in the tree and that the recoloring is extremely regular except at the ends of the path. The same is true of the recoloring following a deletion. We postpone the regular part of this recoloring, indicating a path to be recolored by certain information stored in the top node of the recoloring path. When necessary, we perform some of the recoloring, shortening or splitting the path over which recoloring must still be done.

To be more precise: in the case of an insertion, if we ignore $O(1)$ update steps at the beginning of the insertion and $O(1)$ update steps at the end, the rest of the insertion consists of recoloring a path and some of the children of nodes on the path. Each node on the path has its color flipped, and the red siblings of nodes on the path become black. We call such a path an *insertion interval*. Rather than actually performing the recoloring, we store in the top node of the interval two pieces of header information: a bit indicating the beginning of an insertion interval and the item in the bottom node of the interval.

In the case of a deletion all but $O(1)$ of the update time consists of recoloring the red children of nodes along a path, which become black. We call such a path a *deletion interval*. Rather than actually performing the recoloring, we store in the top node of the interval two pieces of header information: a bit indicating the beginning of a deletion interval, and the item in the bottom node of the interval.

We call insertion intervals and deletion intervals *recoloring intervals*. Our objective is to maintain, by judicious recoloring, the invariant that the recoloring intervals are vertex-disjoint. We shall show that this can be done using $O(1)$ node recolorings per insertion or deletion. Let us see what the disjointness of the recoloring intervals implies. Since we always enter a recoloring interval from the top, we always know whether we are in an interval and when we are entering or leaving one. (When entering an interval, we remember the extra item stored in the top node, which allows us to tell when we leave the interval.) Thus when we are in an interval we can keep track of the current correct node colors, which are not necessarily the same as their actual (unchanged) colors. This gives us the information we need to perform insertions and deletions.

To keep the recoloring intervals disjoint, we adopt the convention that the top and bottom nodes of an insertion interval are actually red (but should be black). This property can be imposed on an insertion interval not having it by coloring $O(1)$ nodes at the ends of the path, thereby shrinking it to an interval with the property. More generally, a recoloring interval can be shrunk by one node at either the top or the bottom by doing $O(1)$ recoloring. It can also be split in the middle into two recoloring intervals by doing $O(1)$ recoloring. Shrinking at the top of an interval requires moving the header information in the top node of the interval down to the new top node. Shrinking at the bottom requires changing the header information in the top node. Splitting requires changing the header information in the top node of the original interval and adding header information to the new top node of the bottom half of the interval. In all cases the update time is $O(1)$.

An unresolved issue is whether the children of nodes on the path defining a recoloring interval should be regarded as part of the interval. We regard the children whose colors are supposed to change as being in the interval and those whose colors are not supposed to change as

not being not in the interval. Thus the red children are in an insertion interval and all children are in a deletion interval. Our last convention is that we require the path defining a deletion interval to contain at least two nodes. We maintain this property by completely recoloring deletion intervals that are too small.

The crucial property of recoloring intervals is that they stop the propagation of color changes during an insertion or deletion. This is what allows us to keep the recoloring intervals disjoint. We verify that color changes stop when they hit a recoloring interval by case analysis, omitted in this version of the paper.

In summary, the lazy recoloring method is as follows. We maintain a red-black tree as a collection of correctly colored nodes not in recoloring intervals and a vertex-disjoint set of intervals that represent the lazy evaluation of very regular color changes. To perform an insertion, we search for the insertion location, keeping track of the bottommost recoloring interval on the way down the tree. We make the necessary changes to place the new red node in the tree. If the red constant is violated, we use the propagating insertion rule to walk up the tree, without actually making the color changes implied. When the insertion stops propagating, we shrink all recoloring intervals in the vicinity sufficiently so that the terminating insertion rules can be applied to correctly colored nodes. This may require splitting at most one recoloring interval, the bottommost along the original search path. We encode the remaining unperformed color changes as a single new insertion interval, disjoint from all other recoloring intervals. The total insertion time is $O(\log n)$, but the update time is only $O(1)$. A deletion is similar. This method is an improvement over Tsakalidis's original scheme (invented to solve a different problem), which has an $O(\log n)$ worst-case update time bound.

Since red-black trees with lazy recoloring have an $O(1)$ worst-case update time bound per insertion or deletion, the node-splitting method of Section 3 makes such trees fully persistent at an amortized space cost of $O(1)$ per insertion or deletion. The worst-case time for an access operation is $O(\log n)$, as is the amortized time of an insertion or deletion. The only competitive method for obtaining full persistence of search trees, due independently to Myers [20,22], Krijnen and Meertens [18], Reps, Teitelbaum, and Demers [26], and Swart [30], requires $O(\log n)$ space and time in the worst case per insertion or deletion. (See [28].) Thus our method saves a logarithmic factor in space.

5. Another Way to Build Fully Persistent Search Trees

In this section we discuss an alternative way to make a search tree fully persistent. As in Section 4 we concentrate on binary search trees, and in particular on red-black trees, although our method applies more generally. Our approach is to combine node copying as refined for the special case of search trees with a new idea, that of *displaced storage of changes*. Instead of

indicating a change to an ephemeral node x by storing the change in the corresponding persistent node $\bar{x}$, we store information about the change in some possibly different node that lies on the access path to $\bar{x}$ in the current version. Thus the record of the change is in general displaced from the node to which the change applies. The path from the node containing the change information to that of the affected node is called the *displacement path*. By copying nodes judiciously, we are able to keep the displacement paths sufficiently disjoint to guarantee an $O(1)$ worst-case space bound per update step and an $O(1)$ worst-case time bound per access step.

To obtain full persistence with this method, we must change the scheme used to navigate through the persistent structure. Instead of linearizing the version tree as in Section 3, we maintain a representation of the version tree that allows queries of the following kind to be answered easily: given two versions i and j , determine whether or not i is an ancestor of j in the version tree. The appropriate representation, discovered by Dietz [10], is a list containing each version twice, once in its preorder position and once in its postorder position with respect to a depth-first traversal of the version tree. We call this list the *traversal list*. If $pre(i)$ and $post(i)$ denote the preorder and postorder occurrences of version i in the traversal list, then i is an ancestor of j in the version tree if and only if $pre(j)$ precedes $pre(i)$ but not $post(i)$ in the traversal list. To update the traversal list when a new version i is created, we insert $pre(i)$ followed by $post(i)$ immediately after $pre(p(i))$. If we maintain the traversal list using the representation of Tsakalidis [34] or that of Sleator [29], then the worst-case time per ancestor-descendant query is $O(1)$ and the amortized time per creation of a version is $O(1)$.

The persistent structure itself consists of a collection of persistent nodes, each of which contains either one or two version records. A version record has exactly the same fields as a node in the ephemeral structure, namely (in the case of a binary tree) an item, a left pointer, a right pointer, and whatever additional fields are used in the particular kind of search tree. (In the case of a red-black tree with lazy recoloring, this information is a color bit, an extra item specifying a recoloring interval, and a bit indicating whether the interval is an insertion or a deletion interval.) The first record in a persistent node is the *original record*; its item is regarded as being associated with the node itself. The second record in a node is the *change record*; it has a version stamp.

The item in the change record of a node need not be the same as the item associated with the node. If these items are different, the change record is said to be *displaced*. Let r be a change record in a node $\bar{x}$, let e be the item in r , and let i be the version stamp of r . The *displacement path* of r is the path taken by a search for e in version i that begins with $\bar{x}$ as the current node and proceeds as follows. If $\bar{y}$ is the current node, i is compared with the version stamp j of the change record in $\bar{y}$. If $\bar{y}$ has no change record, if the change record in $\bar{y}$ is

displaced, or if i is not a descendant of j in the version tree, then the search follows the left or right pointer in the original version record of $\bar{y}$, depending on whether e is less than or greater than the item associated with $\bar{y}$. Otherwise ($\bar{y}$ has a nondisplaced change record with a version stamp that is an ancestor of i in the version tree), the search follows the left or right pointer in the change record, again depending on whether e is less than or greater than the item associated with $\bar{y}$. The search stops when it reaches a node $\bar{z}$ whose associated item is e . Node $\bar{z}$ is the last node on the displacement path; it is the one that should actually contain record r . Node $\bar{x}$ is called the *head* of the displacement path and node $\bar{z}$ is called the *tail*; the *body* of the path consists of all nodes on the path except the head, including the tail.

The third and last part of the data structure is an access array containing a pointer for each version to the node representing the root of the corresponding ephemeral tree. The pointer for the appropriate version is installed after each update operation, as in Sections 2 and 3.

The crux of the method is the maintenance of the displacement paths. The following three invariants on the data structure are maintained:

- (i) Every change record r has a displacement path, i.e. the search described above eventually reaches a node whose associated item is the one in r . (Record r is displaced if and only if this path has a non-empty body.)
- (ii) If r is a change record, every node in the body of its displacement path contains a change record whose version stamp is not a descendant in the version tree of the version stamp of r .
- (iii) Let r be a change record, and let s be another change record in a node on the displacement path of r whose version stamp is an ancestor of the version stamp of r . Then the body (if any) of the displacement path of s is disjoint from the displacement path of r .

We navigate through the structure as follows. Suppose we wish to access item e in version i . We start a search for e at the root node of version i . During the search, we are at a current node $\bar{x}$ and we may have in hand a displaced record r . The general step of the search is as follows. If $\bar{x}$ has e as its associated item, the search terminates. Otherwise, we examine the original and change records in $\bar{x}$ and the record in hand r . Of the records among these three whose item is the same as the item associated with $\bar{x}$, we select the one whose version stamp is an ancestor of i in the version tree; if more than one record qualifies, we select the one whose version stamp is the nearest ancestor of i . We follow the left or right pointer of this version record, depending upon whether e is less than or greater than the item associated with $\bar{x}$. We compute the new displaced record in hand as follows. Let s be the change record in the old current node $\bar{x}$ and let $\bar{y}$ be the new current node. The new

displaced record in hand is r (the old record in hand) if $\bar{y}$ is on the displacement path of r , or s if $\bar{y}$ is on the displacement path of s and the version stamp of s is an ancestor of i . In any other case there is no new displaced record in hand. Invariants (ii) and (iii) imply that not both r and s can qualify to be the new record in hand. The worst-case time per access step is $O(1)$.

For the purpose of simulating update steps, two operations that shorten displacement paths are useful. The first is *shrinking* a displacement path by removing its tail. Let r be a displaced change record in a node $\bar{x}$ and let $\bar{z}$ be the tail of the displacement path of r . Let i be the version stamp of r , $\bar{y}$ the node preceding $\bar{z}$ in the displacement path of r , and s the record in $\bar{y}$ whose pointer to $\bar{z}$ is followed in traversing the displacement path of r . To shrink the displacement path of r , we create a new node $\bar{z}'$, regarded as a copy of $\bar{z}$, add s to $\bar{z}'$ as its original record, create a copy of s' identical to s except that the pointer to $\bar{z}$ is replaced by a pointer to $\bar{z}'$, and insert s' in place of r as the change record in $\bar{x}$. Record s' gets version stamp i . After the shrinking, record r is no longer displaced; record s' is displaced unless $\bar{y} = \bar{z}$, and it has a displacement path equal to the old path of r minus the tail. Node $\bar{z}$ has no change record; thus the shrinking introduces a new place to put a change record.

The second, slightly more complicated operation on a displacement path is *splitting*. Let r be a displaced record with version stamp i in a node $\bar{x}$, let $\bar{z}$ be a node in the body of the displacement path other than the tail, let $\bar{y}$ be the predecessor of $\bar{z}$ along the displacement path of r , let s be the record in $\bar{y}$ whose pointer to $\bar{z}$ is followed in traversing the displacement path, and let t be the record in $\bar{z}$ whose pointer is followed in continuing along the displacement path from $\bar{z}$. To split the path at $\bar{y}$, we create a new node $\bar{z}'$, regarded as a copy of $\bar{z}$, containing a copy of t as its original record and r with version stamp i as its change record. We also create a copy s' of s , replacing the pointer to $\bar{z}$ by a pointer to $\bar{z}'$, and insert s' in the place of r as the change record of $\bar{x}$. Record s' gets version stamp i . This splits the old displacement path of r in two; the top half, from $\bar{x}$ to $\bar{y}$, is now the displacement path of s' ; the bottom half, from $\bar{z}'$ (instead of $\bar{z}$) to the old tail is the new displacement path of s . Both shrinking and splitting preserve invariants (i)-(iii).

We are now ready to discuss the simulation of ephemeral update steps. For simplicity, we shall assume that each update operation changes only one ephemeral node. The method is easily modified to handle any number of changes per update operation, and in particular a constant number, as in red-black trees with lazy recoloring. Suppose we wish to simulate an ephemeral update that converts version $p(i)$ into version i by changing node x . Let $\bar{x}$ be the persistent node corresponding to x . We construct a change record r with version stamp i containing the field values representing the new version of x . To find a place to store r , we back up along the access path to $\bar{x}$ in version $p(i)$ until we back up past the root of version $p(i)$ or we reach a node $\bar{y}$ has no change

record or $\bar{y}$ is in the body of the displacement path of another change record s whose version stamp is an ancestor of $p(i)$ and such that s is in a node, say $\bar{z}$, on the access path to $\bar{x}$ in version $p(i)$. There are four cases:

- (1) We back up past the root. In this case we create a new root node for version i . The new node contains as its original record a copy of the record in the old root whose pointers are followed in version $p(i)$. As its change record, the new node contains r .
- (2) Node $\bar{y}$ contains no change record. We merely store r in $\bar{y}$ as its change record.
- (3) Node $\bar{y}$ is the tail of the displacement path of s . We shrink the displacement path of s and store r in the newly created copy $\bar{y}'$ of $\bar{y}$ as its change record.
- (4) Node $\bar{y}$ is in the body of the displacement path of s but is not the tail. We split the displacement path of s at $\bar{y}$ and proceeded as in case (b), since after the split node $\bar{y}$ is the tail of the displacement path of the record replacing s in $\bar{z}$.

In all cases the number of nodes added to the persistent structure is $O(1)$ and invariants (i)-(iii) are preserved. If we apply this method to red-black trees with lazy recoloring, the worst-case time for an insertion or deletion is $O(\log n)$, including the time necessary to find the insertion or deletion position but not including the time necessary to update the list representing the version tree. The space used per insertion or deletion is $O(1)$. Thus we obtain a fully persistent form of red-black trees with the same resource bounds as in Section 4, but the $O(1)$ space bound per insertion/deletion is worst-case rather than amortized. The data structure is also somewhat simpler. As in Sections 2 and 3, the method can be modified so that changes are associated with individual fields rather than with complete sets of fields, and the number of change records or change fields per node can be varied to improve the space usage by a constant factor.

6. Applications, Extensions, and Open Problems

In this concluding section we mention some applications and extensions of our work and some open problems. We first note some of the kinds of persistent data structures that can be obtained using our techniques.

- (i) An ephemeral stack or queue can be implemented as a singly-linked list with $O(1)$ -time insertion and deletion of items (at the appropriate ends) [31]. The method of Section 2 makes these data structures partially persistent at a space and time cost of $O(1)$ per operation; the method of Section 3 gives the same bounds for fully persistent stacks and queues.
- (ii) An ephemeral deque (double-ended queue) can be implemented as a doubly-linked list with $O(1)$ -time insertion and deletion of items at either end [31]. We can make this structure partially or fully

persistent using the method of Section 2 or 3, respectively, at a time and space cost of $O(1)$ per operation.

- (iii) As discussed in Section 4 and 5, we can obtain a partially or fully persistent red-black tree with logarithmic access, insertion, and deletion time and $O(1)$ space cost per insertion or deletion.
- (iv) The techniques of Sections 4 and 5 can be extended to obtain partially or fully persistent red-black trees with fingers, in which the time to access, insert, or delete an item d positions away from a finger is $O(\log d)$ and the time to move a finger d positions is $O(\log d)$. The space for an insertion or deletion is $O(1)$. For a finger movement, the space bound is $O(1)$ in the case of partial persistence and $O(\log d)$ in the case of full persistence. To obtain full persistence, an extension of the lazy recoloring method of Section 4 is needed.

The main applications of these data structures are in computational geometry, text and file editing, and implementation of very high level languages. As discussed in the companion paper [28], partially persistent search trees can be used to give a simple solution to the planar point location problem for a polygonal subdivision with a query time of $O(\log n)$, a space bound of $O(n)$, and a preprocessing time bound of $O(n \log n)$. They can also be used as a simpler substitute for Chazelle's "hive graph" structure, which has a number of uses in geometric retrieval [5]. Cole [9] lists a number of other geometric applications of such search trees.

Fully persistent search trees can be used to represent any sorted set or list as it evolves over time, allowing updates in any version. Such structures have been used in text editing by Myers [20,22], in program editing by Reps, Teitelbaum, and Demers [26], and in implementation of high-level data structures in the programming language *B* by Krijnen and Meertens [18]. Fully persistent deques and search trees can also be used to implement lists and sets in SETL and related very high level programming languages.

Among open problems related in our work, the following three are especially significant:

- (i) Find a way to make the time and space per update step $O(1)$ in the worst case instead of in the amortized case. The method of Section 5 for obtaining fully persistent red-black trees, if used to obtain partially persistent trees, uses $O(1)$ time and space in the worst case per insertion or deletion, not counting the $O(\log n)$ time needed to search for the location of the insertion or deletion. The difficulty in the fully persistent case is that the data structure for implementing the traversal list has only an amortized $O(1)$ time bound per insertion; this bound must be made worst-case. For the general methods of Sections 2 and 3, the issue is whether node copying or splitting can somehow be delayed or otherwise modified so that there is $O(1)$

node copying or splitting in the worst case per update step.

- (ii) Find a way to allow update operations that combine two or more old versions of the structure. This would allow concatenation of lists, for example. The difficulty here is that the navigation problem becomes much more difficult. In the fully persistent case, the version tree of Section 3 becomes a directed acyclic graph (the predecessors of each version are the versions from which it is formed).
- (iii) Find a way to allow update operations that change many reasons simultaneously. for example, consider the case of multiple versions of a binary search tree, indexed by integers, in which each insertion or deletion affects all versions with indices in some interval. The dynamization techniques of Bentley and Saxe [2] suggest an approach to this problem, that works reasonably well in the case of insertions, but deletions seem much harder to handle.

REFERENCES

- [1] R. Bayer and E. McCreight, "Organization of large ordered indexes," *Acta Informatica* 1 (1972), 173-189.
- [2] J. L. Bentley and J. B. Saxe, "Decompositional searching problems I: static-to-dynamic transformations," *J. Algorithms* 1 (1980), 301-358.
- [3] N. Blum and K. Mehlhorn, "On the average number of rebalancing operations in weight-balanced trees," *Theoretical Computer Science* 11 (1980), 303-320.
- [4] M. R. Brown and R. E. Tarjan, "Design and analysis of a data structure for representing sorted lists," *SIAM J. Comput.*, 9 (1980), 594-614.
- [5] B. Chazelle, "Filtering search: a new approach to query-answering," *Proc. 24th Annual IEEE Symp. on Foundations of Computer Science* (1983), 122-132.
- [6] B. Chazelle, "How to search in history," *Information and Control*, to appear.
- [7] B. Chazelle and L. J. Guibas, "Fractional cascading: a data structuring technique with geometric applications (extended abstract)," *Automata, Languages, and Programming*, 12th Colloquium, W. Brauer ed., *Lecture Notes in Computer Science*, 194, Springer-Verlag, Berlin (1985) 90-100.
- [8] B. Chazelle and L. J. Guibas, "Fractional cascading I: a data structuring technique," to appear.
- [9] R. Cole, "Searching and storing similar lists," *J. Algorithms*, to appear.
- [10] P. Dietz, "Maintaining order in a linked list," *Proc. 14th Annual ACM Symp. on Theory of Computing* (1982), 62-69.
- [11] D. P. Dobkin and J. I. Munro, "Efficient uses of the past," *Proc. 21st Annual IEEE Symp. on Foundations of Computer Science* (1980), 200-206.
- [12] L. J. Guibas and R. Sedgwick, "A dichromatic framework for balanced trees," *Proc. 19th Annual IEEE Symp. on Foundations of Computer Science* (1978), 8-21.
- [13] R. Hood and R. Melville, "Real-time queue operations in pure LISP," *Inform. Process. Lett.* 13 (1981), 50-54.
- [14] S. Huddleston and K. Mehlhorn, "Robust balancing in B-trees," *Theoretical Computer Science VII*, P. Deussens, ed., *Lecture Notes in Computer Science* 104, Springer-Verlag, Berlin (1981), 234-244.
- [15] S. Huddleston and K. Mehlhorn, "A new data structure for representing sorted lists," *Acta Informatica* 17 (1982), 157-184.
- [16] S. Huddleston, "An efficient scheme for fast local updates in linear lists," Dept. of Information and Computer Science, University of California, Irvine, CA, 1981.
- [17] S. R. Kosaraju, "Localized search in sorted lists," *Proc. 13th Annual ACM Symp. on Theory of Computing* (1981), 62-69.
- [18] T. Krijnen and L. G. L. T. Meertens, "Making B-trees work for B," IW 219/83, The Mathematical Centre, Amsterdam, The Netherlands, 1983.
- [19] D. Maier and S. C. Salveter, "Hysterical B-trees," *Inform. Process. Lett.* 12 (1981), 199-202.
- [20] E. W. Meyers, "AVL dags," TR 82-9, Dept. of Computer Science, The University of Arizona, Tucson, AZ, 1982.
- [21] E. W. Myers, "An applicative random-access stack," *Inform. Process. Lett.* 17 (1983), 241-248.
- [22] E. W. Myers, "Efficient applicative data types," *Conf. Record Eleventh Annual ACM Symp. on Principles of Programming Languages* (1984), 66-75.
- [23] J. Nievergelt and E. M. Reingold, "Binary search trees of bounded balance," *SIAM J. Comput.* 2 (1973), 33-43.
- [24] M. H. Overmars, "Searching in the past I," *Information and Control*, to appear.
- [25] M. H. Overmars, "Searching in the past II: general transforms," Technical Report RUU-CS-81-9, Department of Computer Science, University of Utrecht, Utrecht, The Netherlands, 1981.
- [26] T. Reps, T. Teitelbaum, and A. Demers, "Incremental context-dependent analysis for language-based editors," *ACM Trans. on Prog. Sys. and Lang.* 5(193), 449-477.
- [27] N. Sarnak, "Persistent data structures," Ph.D. Thesis, Dept. of Computer Science, New York University, New York, NY, to appear.

- [28] N. Sarnak and R. E. Tarjan, "Planar point location using persistent search trees," *Comm. ACM*, to appear.
- [29] D. D. Sleator, "An improved method for maintaining order in a list," to appear.
- [30] G. F. Swart, "Efficient algorithms for computing geometric intersections," Technical Report #85-01-02, Department of Computer Science, University of Washington, Seattle, WA, 1985.
- [31] R. E. Tarjan, *Data Structures and Network Algorithms*, Society for Industrial and Applied Mathematics, Philadelphia, PA, 1983.
- [32] R. E. Tarjan, "Updating a balanced search tree in $O(1)$ rotations," *Inform. Process. Lett.* 16 (1983), 253-257.
- [33] R. E. Tarjan, "Amortized computational complexity," *SIAM J. Alg. Disc. Meth.* 6 (1985), 306-318.
- [34] A. K. Tsakalidis, "Maintaining order in a generalized linked list," *Acta Informatica* 21 (1984), 101-112.
- [35] A. K. Tsakalidis, "AVL-trees for localized search," *Automata, Languages, and Programming*, 11th Colloquium, J. Paredaens, ed., *Lecture Notes in Computer Science* 172, Springer-Verlag, Berlin (1984), 473-485.
- [36] A. K. Tsakalidis, "An optimal implementation for localized search," A 84/06, Fachbereich Angewandte Mathematik und Informatik, Universität des Saarlandes, Saarbrücken, West Germany, 1984.

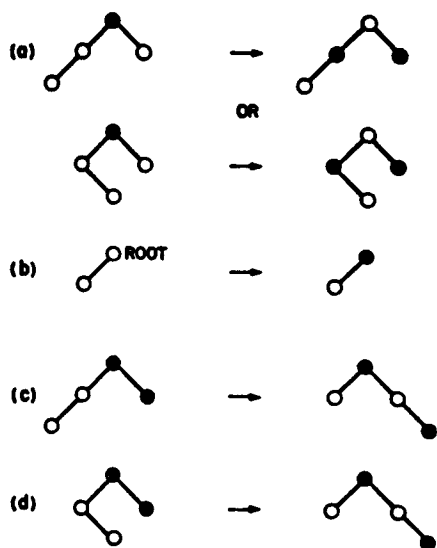


Figure 1

The rebalancing transformations in red-black tree insertion. Symmetric cases are omitted. Solid nodes are black; hollow nodes are red. All unshown children of red nodes are black. In cases (c) and (d) the bottommost black node shown can be missing.

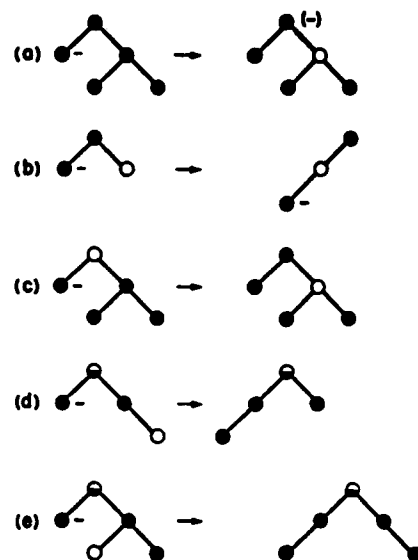


Figure 2

The rebalancing transformations in red-black tree deletion. The two ambiguous (half-solid) nodes in (d) have the same color, as do the two in (e). Minus signs denote short nodes. In (a), the top node after the transformation is short unless it is the root.